

BASH

THE BOURNE-AGAIN SHELL

“Горячие” клавиши

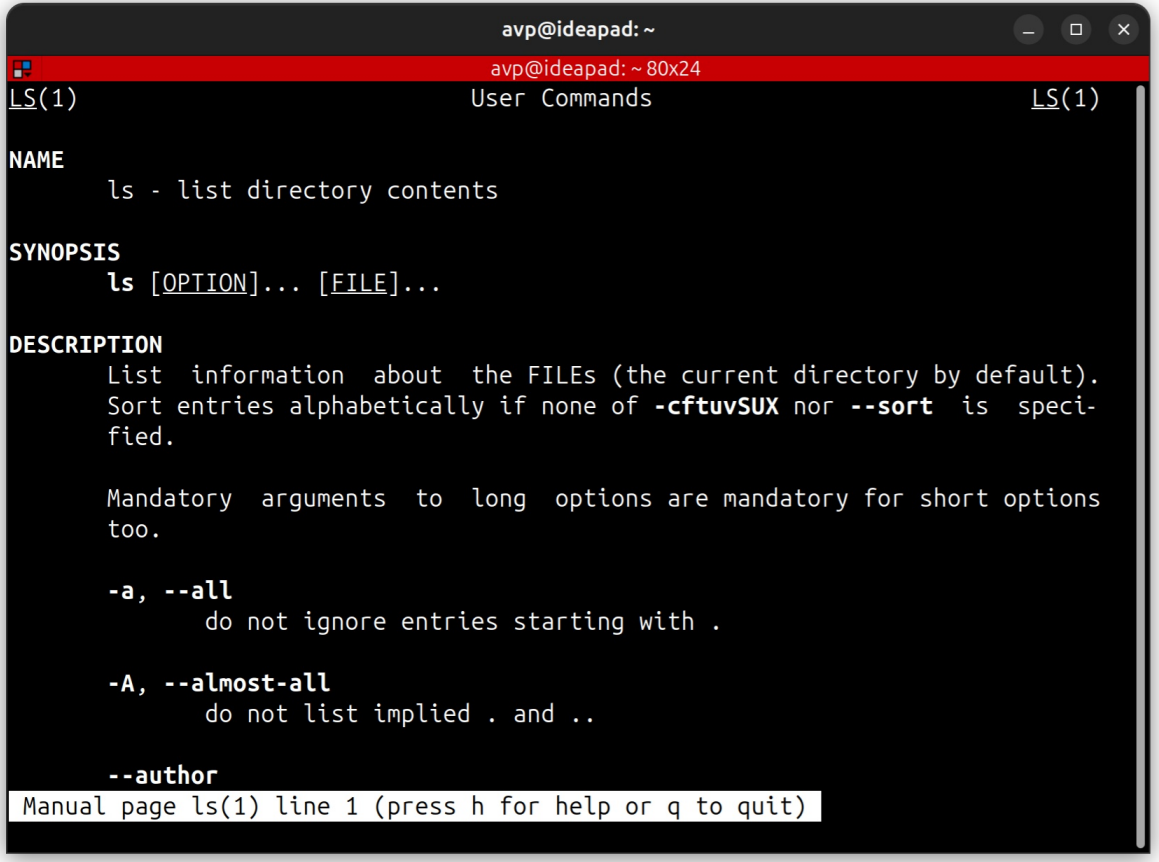
- **Ctrl+f** – Move the cursor one character to the right
- **Ctrl+b** – Move the cursor one character to the left
- **Alt+f** – Move the cursor one word to the right
- **Alt+b** – Move the cursor one word to the left
- **Ctrl+a** – Move the cursor to the beginning of the current commandline
- **Ctrl+c** – Cancels the current command and presents a new prompt
- **Ctrl+d** – Closes the current Bash instance, possibly also closing the terminal-emulator
- **Ctrl+e** – Move the cursor to the end of the current commandline
- **Ctrl+q** – Wake the terminal; buffered keypresses are then processed
- **Ctrl+s** – Put the terminal to sleep
- **Ctrl+w** – Remove one word to the left of the cursor
- **Ctrl+z** – Stop a foregrounded process

Получение справки

- Встроенная справка по команде:
  ```
  $ ls --help
  $ ls -h
  ```
- Открытие справочной страницы:
  ```
  $ man ls
  ```

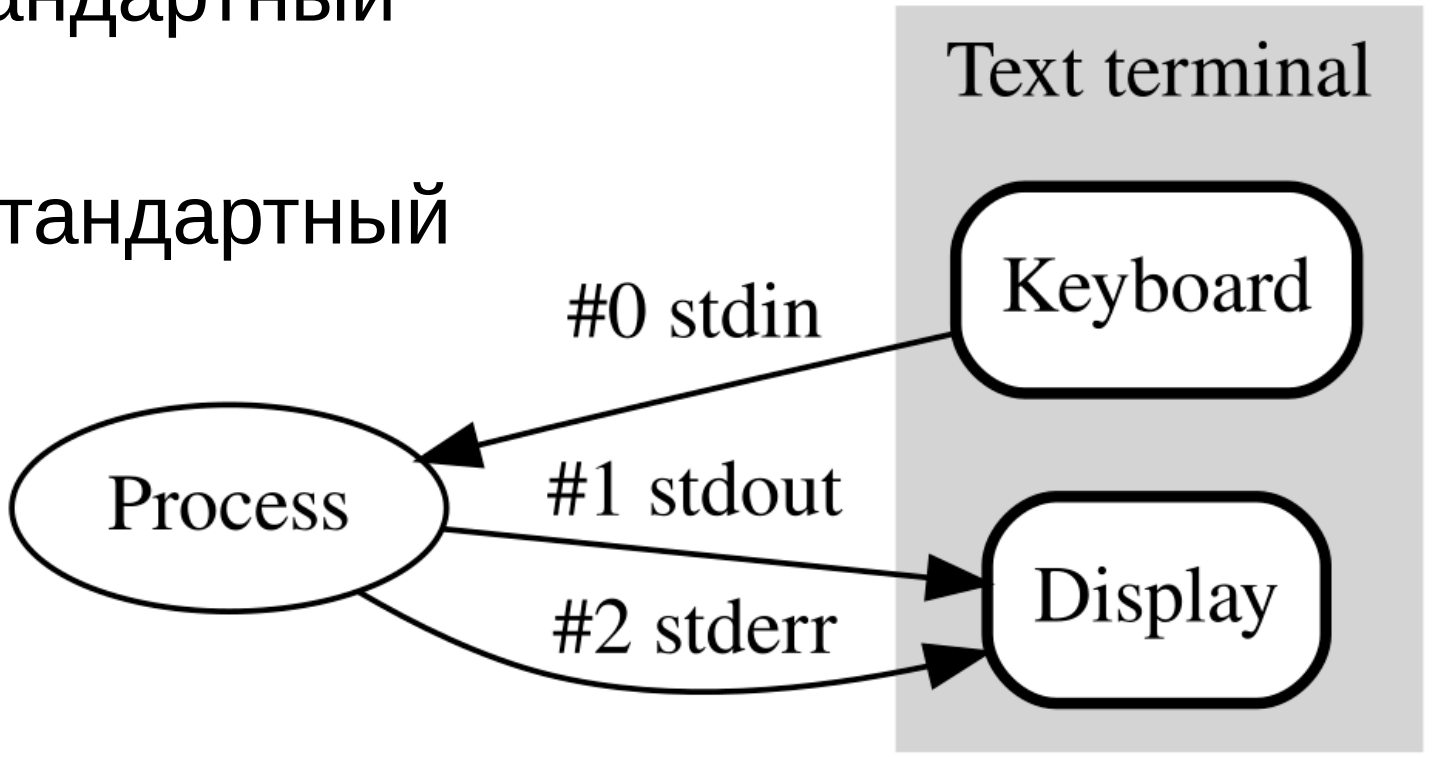
  Выбор раздела справки:
  ```
  $ man 3 printf
  ```

  Поиск по справке:
  ```
  $ man -k printf
  ```
- Гипертекстовая справка:
  ```
  $ info nano
  ```



Потоки ввода-вывода

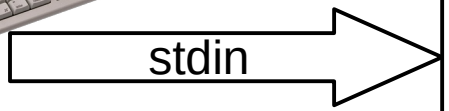
- **stdin** – стандартный ввод
- **stdout** – стандартный вывод
- **stderr** – вывод ошибок

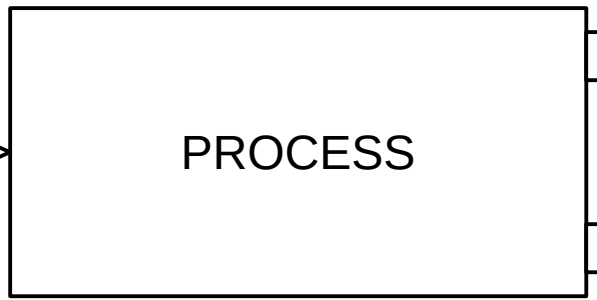


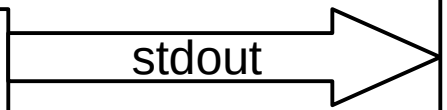
Перенаправление потоков

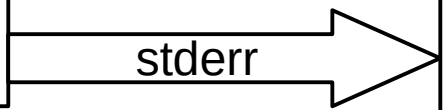
```
$ program > output.txt
```

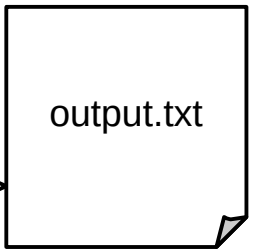








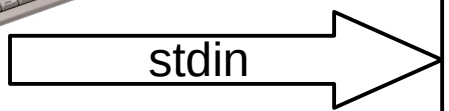


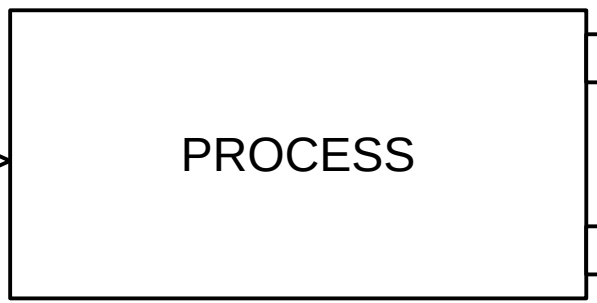


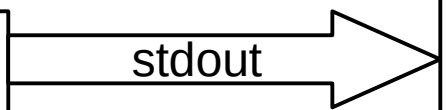


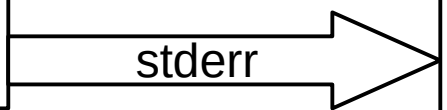

```
$ program 2> errors.txt
```



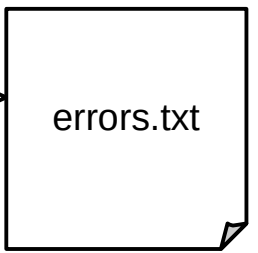






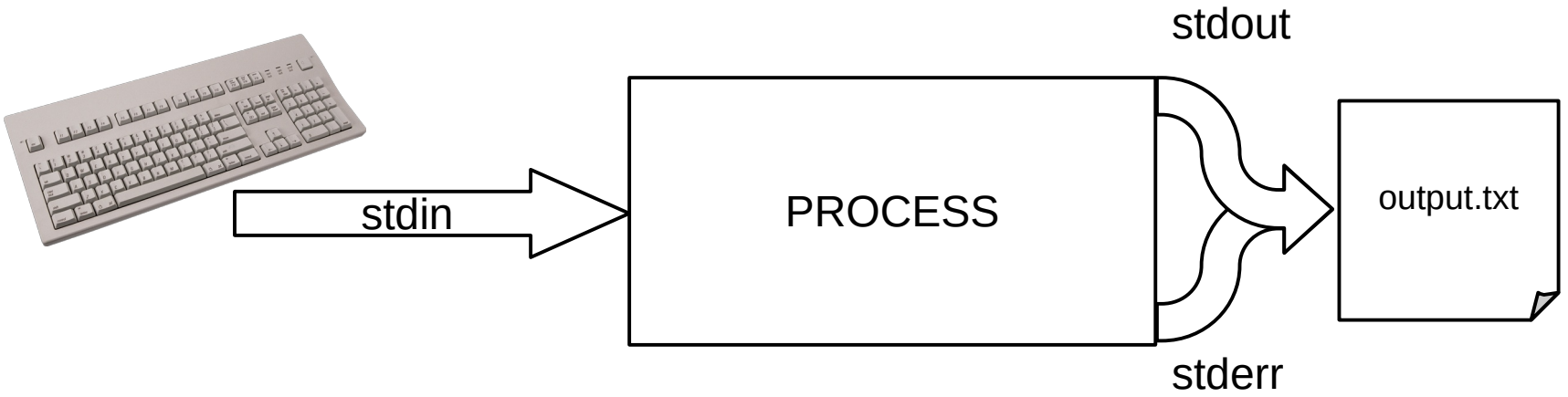






Объединение потоков

```
$ program > output.txt 2>&1
```



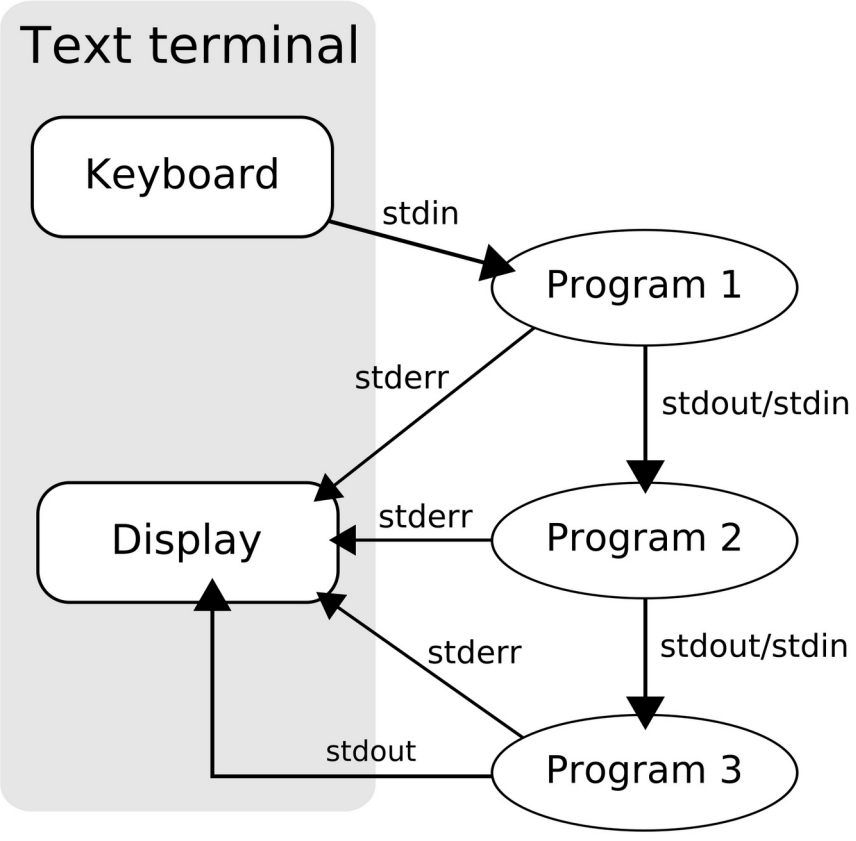
Конвейер

Параллельное выполнение:

```
$ program-1 | program-2
```

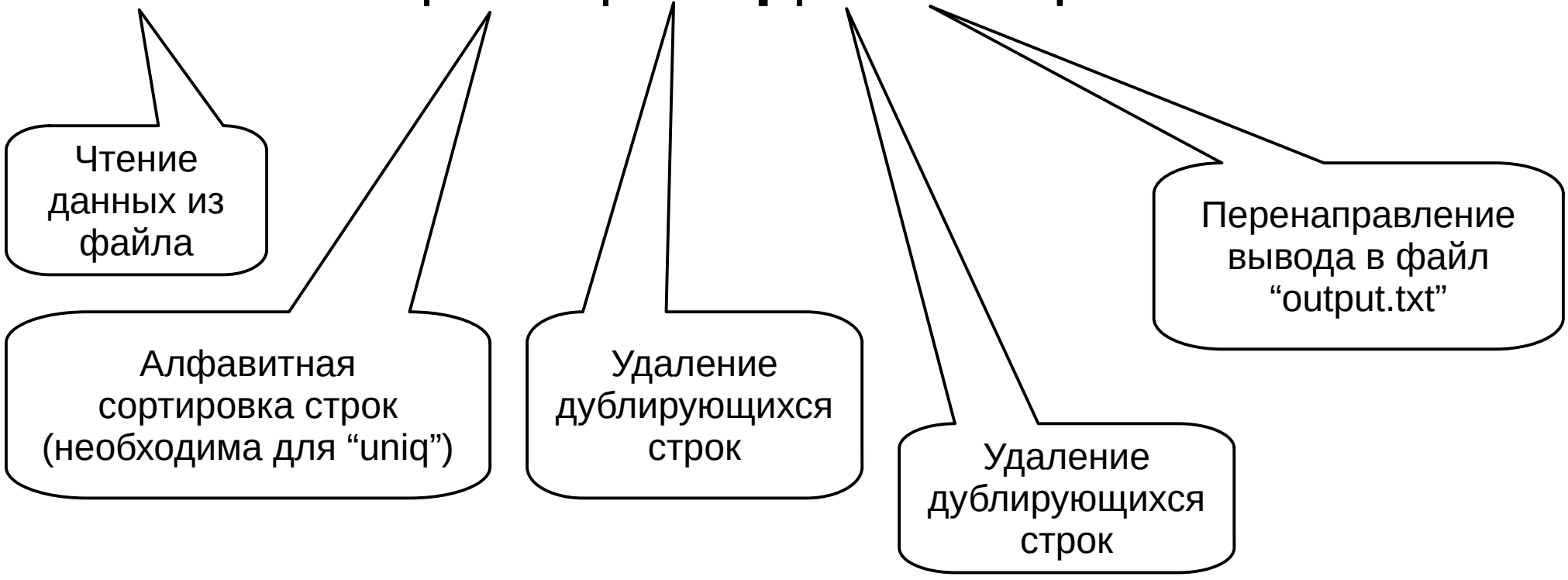
Последовательное выполнение:

```
$ program-1; program-2
$ program-1 && program-2
$ program-1 || program-2
```



Пример конвейера

$ **cat** data.txt | **sort** | **uniq** | **nl** > output.txt



Логические операции

Логическое “ИЛИ”:

```
$ program-1 || program-2
```

Логическое “И”:

```
$ program-1 && program-2
```

Пример совмещения команд:

```
$ program-1 || echo “ERROR”
$ program-1 && echo “SUCCESS” || echo “ERROR”
```

Команда “test”

```
$ test -e “file.txt” && \
    echo “File exists”

$ test -d “file” && \
    echo “It is a directory” \
    || echo “It is not a directory”
```

Статус завершения программы

Статус 0 означает “успех”, или “истина” *

```
$ exit 0
```

Значение больше нуля означает “ошибка”, или “ложь”

```
$ exit 10
```

Смысл не-нулевых значений отличается в разных программах – см. справку:

```
$ man ls
```

* Однако в языке C ноль означает “ложь”

```
int main(int argc, char** argv) {
    return 0;
}
```

Переменные окружения

Объявление переменной:

```
$ A=10
```

Вывод значения на экран:

```
$ echo $A
```

Удаление переменной:

```
$ A=
$ unset A
```

Экспортирование переменной:

```
$ A=10
$ export A
```

Или:

```
$ export A=10
```

Запуск программы с переменной:

```
$ A=10 program
```

Подстановка переменных – 1

```
$ NAME=”avp”
$ echo “Hello, $NAME”

$ FILE=”some-file.txt”
$ rm $FILE

$ COMMAND=ls
$ $COMMAND -a
```

Более сложный случай:

```
$ PREFIX=”test”
$ touch ${PREFIX}file
```

Подстановка переменных – 2

```
$ FILE=”/usr/bin/ program”
$ rm -rf $FILE
```

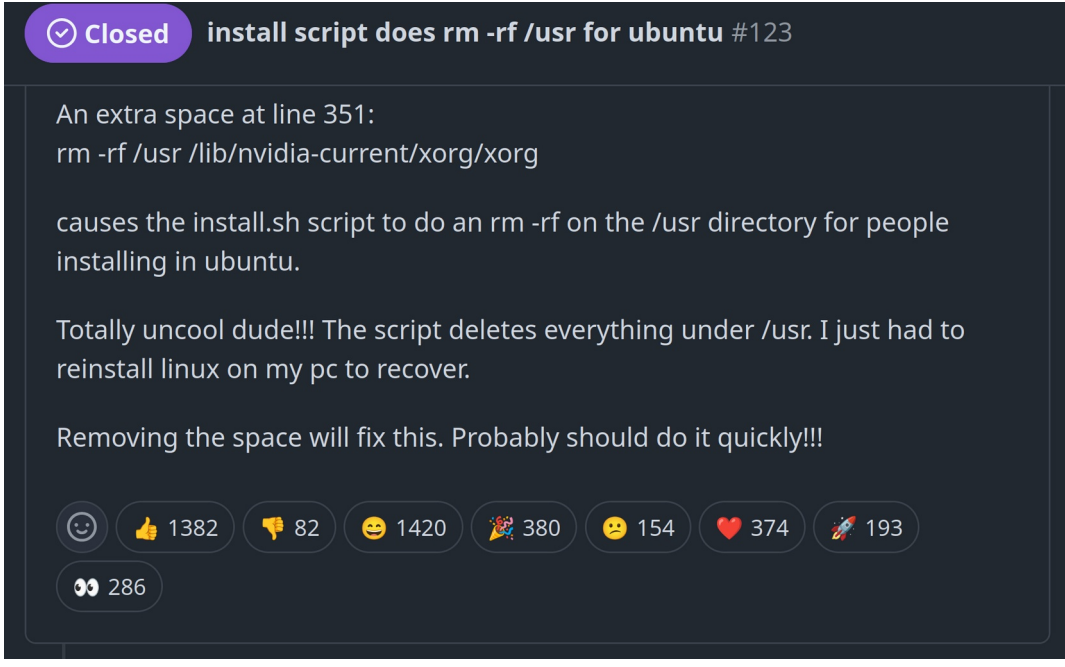
Подстановка переменных – 3

Ошибка!

```
$ FILE=”/usr/bin/ program”
$ rm -rf $FILE
```

Правильный вариант:

```
$ FILE=”/usr/bin/ program”
$ rm -rf “$FILE”
```



Мораль: всегда выполняйте распаковку переменных внутри кавычек!

https://github.com/MrMEEE/bumblebee-Old-and-abbandoned/issues/123

Специальные переменные

```
Статус завершения последней команды:
$ echo $?


Рабочий каталог процесса:
$ echo $PWD

Текущий интерпретатор (оболочка):
$ echo $SHELL

Текущий пользователь:
$ echo $USER

Путь поиска программ:
$ echo $PATH
```

Управляющие конструкции

Условие

```
if условие; then
    действие
    ...
else
    действие
    ...
fi
```

Цикл “while”

```
while условие; do
    тело цикла
done
```

Цикл “for”

```
for a in список; do
    тело цикла
done
```

Команда “seq”

```
$ seq 5
1
2
3
4
5
```

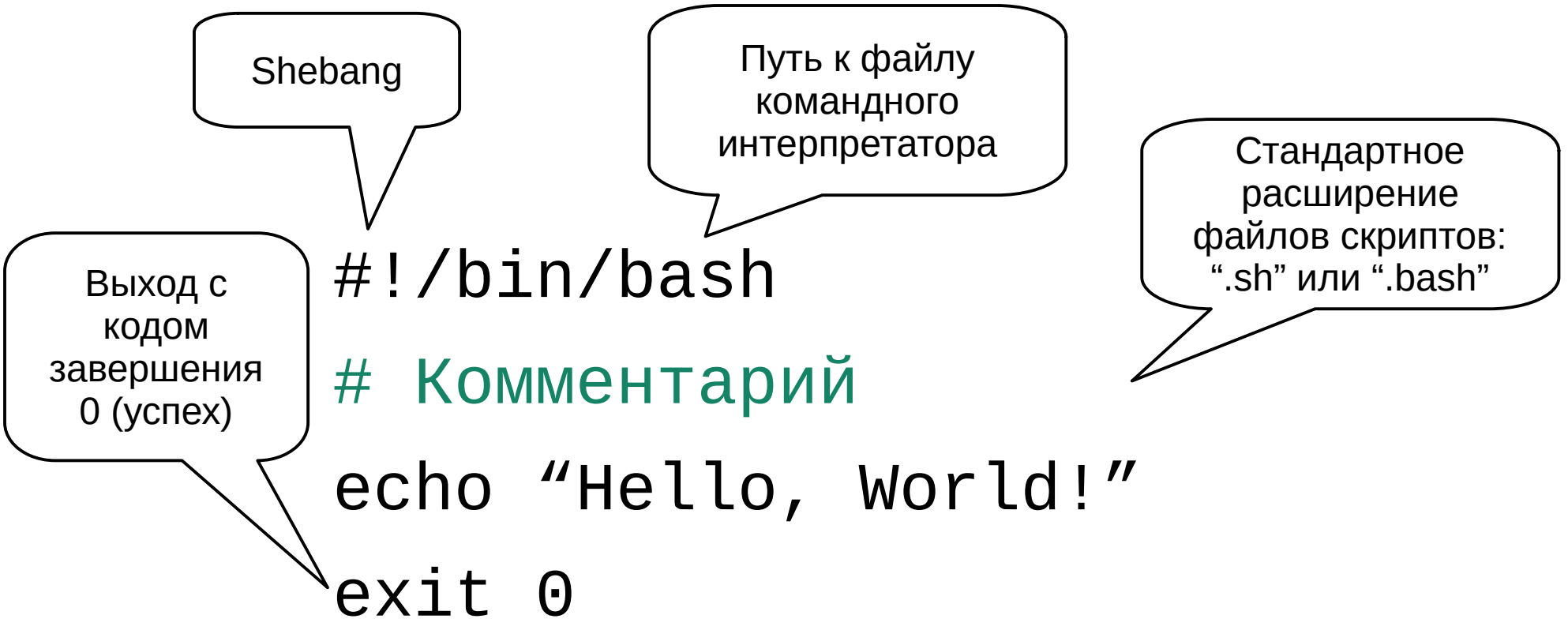
```
$ seq 5 8
5
6
7
8
```

```
$ seq 5 2 10
5
7
9
```

Использование “seq” в циклах

```
for n in $(seq 4); do
    echo “Line number $n”
done
```

Не-интерактивный режим



Исполнение скрипта

Через явное указание интерпреператора:

```
$ bash script.sh
```

Запуск с интерпретатором, указанным в shebang:

```
$ chmod +x script.sh
$ ./script.sh
```

Получение аргументов программы

```
#!/bin/bash

# Вывод количества аргументов:
echo $#

# Вывод первого аргумента:
echo $1

# ВАЖНО: Нулевой аргумент содержит имя программы.

# Вывод всех аргументов:
for a in $*; do
    echo $a
done
```

Функции

```
hello() {
    echo “Hello, $1”
}

hello “avp”
```

Задачи для практики

- Разработать скрипт резервного копирования. Скрипт должен создавать сжатый архив с данными из указанного каталога. Имя каталога должно передаваться аргументом командной строки. Имя файла архива должно иметь суффикс в виде текущей даты и времени с точностью до секунды.
- Доработать скрипт так, чтобы была ротация резервных копий – например, хранились последние 10 копий. При создании новой копии, самая старая должна удаляться.
- Сделать автоматический запуск скрипта с помощью CRON.

Источники изображений

- GNU Bash logo © Free Software Foundation — Free Art License: https://commons.wikimedia.org/wiki/File:Gnu-bash-logo.svg
- POSIX standard streams diagram © Danielpr85, TuukkaH — Public Domain: https://commons.wikimedia.org/wiki/File:Stdstreams-notitle.svg
- Unix pipeline diagram © XcepticZP, TuukkaH — Public Domain: https://commons.wikimedia.org/wiki/File:Pipeline.svg